call, apply, bind & Currying

9 min read · Updated 17 Jul 2026 · Topic 10 of 23 · Bookmark · Completed

`call`, `apply`, and `bind` are the three methods every function inherits from `Function.prototype` that let you set `this` explicitly instead of leaving it to the call-site. They are the backbone of method borrowing, partial application, and fixing the classic "lost `this`" bug in callbacks — which is exactly why interviewers reach for them. Currying and function composition build on the same idea (functions that produce functions) and round out the functional-programming portion of a JS interview.

`call` vs `apply` vs `bind`

All three answer one question: **what should `this` be when this function runs?** The difference is timing and how arguments are passed.

- `call(thisArg, arg1, arg2, ...)` — invokes the function **immediately**, arguments passed as a **comma list**.
- `apply(thisArg, [arg1, arg2])` — invokes the function **immediately**, arguments passed as a **single array**.
- `bind(thisArg, arg1, ...)` — does **not** invoke; returns a **new function** permanently bound to `thisArg` (and optionally some leading args), to call later.

A memory hook: **a** in `apply` is for **a**rray; `call` is for **c**omma-separated; `bind` **b**akes it in for later.

Here is one shared example — a method living on `person` borrowed by `me`:

JS Copy

```
const person = {
  fullName: function (city, country) {
    return `${this.first} ${this.last} from ${city}, ${country}`;
```

```
const me = { first: "Ada", last: "Lovelace" };

person.fullName.call(me, "London", "UK");   // "Ada Lovelace from London,
person.fullName.apply(me, ["London", "UK"]); // "Ada Lovelace from London,

const bound = person.fullName.bind(me, "London"); // not called yet
bound("UK");                                      // "Ada Lovelace from London,
```

Method	Invokes now?	Args form	Returns
`call`	yes	comma list	the function's result
`apply`	yes	single array	the function's result
`bind`	no	comma list (partial)	a new bound function

Method borrowing

Because `this` is decided at call-time, you can "borrow" a method from one object and run it against another — you don't need inheritance. The two interview-classic examples:

Borrowing `Array.prototype.slice` to turn `arguments` into a real array. The old-school `arguments` object is array-like (has indices and `length`) but isn't an `Array`, so it has no `.slice`/`.map`. Borrowing `slice` with `call` copies it into a genuine array:

JS Copy

```
function argsDemo() {
  return Array.prototype.slice.call(arguments);
}
argsDemo(1, 2, 3); // ⇒ [1, 2, 3]  (a real array)
```

Borrowing `Math.max` with `apply`. `Math.max` takes a variadic list, not an array. Before spread existed, `apply` was the way to feed it an array (the `thisArg` is irrelevant here,

so pass `null`):

JS Copy

```
const nums = [5, 6, 2, 9, 1];
Math.max.apply(null, nums); // ⇒ 9
Math.max(...nums);          // ⇒ 9  (modern equivalent)
```

Modern JS often replaces both with spread (`[...arguments]`, `Math.max(...nums)`) and rest params — a great thing to mention in an interview: *"`apply` for method borrowing has largely been superseded by the spread operator, but the mechanism still matters for `bind` and for understanding `this`."*

`bind` for partial application and fixing `this`

`bind` does two jobs at once. First, **partial application**: the args you pass to `bind` are pre-filled ("baked in"), and later args append after them.

JS Copy

```
function multiply(a, b) {
  return a * b;
}
const double = multiply.bind(null, 2); // a is fixed to 2
double(8); // ⇒ 16
```

Second, **pinning `this` for a callback**. When you pass a method as a callback (`setTimeout`, event handlers, `.map`), the method is detached from its object and `this` is lost. `bind` locks it back on:

JS Copy

```
const counter = {
  count: 0,
  inc() {
    this.count++;
  },
};
const boundInc = counter.inc.bind(counter);
```

A bound function cannot be re-bound. Once `bind` fixes `this`, a second `bind` (or a later `call` / `apply`) cannot change it — the first binding wins:

JS Copy

```
function who() {
  return this.name;
}
const a = who.bind({ name: "A" });
const b = a.bind({ name: "B" }); // second bind is ignored for `this`
b(); // ⇒ "A"
```

Polyfill: `Function.prototype.myBind`

Implementing `bind` is one of the most-asked JS interview tasks. The key insight: `bind` must (1) capture the original function via `this`, (2) remember the pre-bound args, and (3) return a new function that merges pre-bound args with call-time args and invokes the original with the right `this`. A robust version uses `apply` internally:

JS Copy

```
Function.prototype.myBind = function (thisArg, ...boundArgs) {
  const fn = this; // the function myBind was called on
  return function (...callArgs) {
    return fn.apply(thisArg, [...boundArgs, ...callArgs]);
  };
};

function greet(greeting, punctuation) {
  return `${greeting}, ${this.name}${punctuation}`;
}
const helloGrace = greet.myBind({ name: "Grace" }, "Hello");
helloGrace("!"); // ⇒ "Hello, Grace!"
```

A senior-level follow-up is *"does your polyfill support `new`?"* The real `bind` ignores the bound `this` when the result is called with `new` and instead uses the freshly constructed instance. Supporting that requires detecting `new` (via `instanceof`) inside the returned function and calling the original as a constructor — worth mentioning even if you write the simpler version above first.

Currying

Currying transforms a function that takes all its arguments at once, `f(a, b, c)`, into a chain of unary calls, `f(a)(b)(c)` — each call takes one argument and returns a function waiting for the next. A generic `curry` helper collects arguments until it has enough (using the original's `.length`, its arity), then invokes:

JS Copy

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return (...next) => curried.apply(this, [...args, ...next]);
  };
}
function sum3(a, b, c) {
  return a + b + c;
}
const cs = curry(sum3);
cs(1)(2)(3);   // => 6
cs(1, 2)(3);   // => 6  (this flexible form accepts groups too)
cs(1)(2, 3);   // => 6
cs(1, 2, 3);   // => 6
```

Partial application vs currying — a frequent clarifying question. They're related but not identical:

- **Partial application** fixes *some* arguments and returns a function taking the *rest* in one call — `bind` does exactly this. `multiply.bind(null, 2)` yields a function of one remaining argument.

So partial application is "pre-fill a few, call once more"; currying is "one argument at a time, all the way down." The flexible `curry` above blurs the line by accepting groups, but true currying is strictly one-at-a-time.

Function composition: `compose` and `pipe`

Composition is the related functional tool: chain single-argument functions so the output of one feeds the next. `compose` runs **right-to-left** (math convention), `pipe` runs **left-to-right** (reading order):

JS Copy

```
const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x
const pipe    = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);

const inc = (n) => n + 1;
const dbl = (n) => n * 2;

compose(inc, dbl)(5); // => 11  (dbl first: 10, then inc: 11)
pipe(inc, dbl)(5);    // => 12  (inc first: 6, then dbl: 12)
```

Curried functions compose cleanly because each stage is unary — which is why currying and composition are usually taught together.

Common interview questions

Q: What's the difference between `call` and `apply`? Both invoke the function immediately with an explicit `this`; `call` takes arguments as a comma-separated list, `apply` takes them as a single array. Mnemonic: **a**pply → **a**rray.

Q: How is `bind` different from `call` / `apply`? `bind` doesn't invoke — it returns a new function permanently bound to the given `this` (and any pre-filled args), to be called later. `call` / `apply` run right away.

`apply`s the source with `[...boundArgs, ...callArgs]`. Bonus points for handling `new`.

Q: Implement a `curry` function. See `curry` above: accumulate args, and once `args.length ≥ fn.length` call the original; otherwise return a collector for more. Know the `fn.length` (arity) trick — it's what tells `curry` when to stop.

Q: Difference between currying and partial application? Partial application fixes some args and takes the rest in one more call (that's `bind`); currying decomposes into a chain of unary calls, one argument each.

Q: How do you convert `arguments` to an array? Classic: `Array.prototype.slice.call(arguments)`. Modern: `[...arguments]` or a rest parameter `(...args)`.

Gotchas / traps

Losing `this` by detaching a method. Assigning `const fn = obj.greet` and calling `fn()` drops the object binding. In **strict mode / ES modules** `this` is `undefined` and `this.name` throws `TypeError`; in **sloppy mode** `this` falls back to the global object and you silently get `"hi undefined"`. Interviewers love this because the failure mode differs by strictness:

JS Copy

```
const obj = { name: "widget", greet() { return `hi ${this.name}`; } };
const fn = obj.greet;
fn(); // sloppy: "hi undefined"  |  strict/module: TypeError
```

Arrow functions ignore `call` / `apply` / `bind` for `this`. Arrows capture `this` lexically at definition, so passing a `thisArg` does nothing — a common trick question:

JS Copy

```
const arrow = () => this;
```

`bind` + `new`. If you construct with a bound function, the bound `this` is discarded (the new instance wins) but **pre-bound arguments are still applied**, and `instanceof` still points to the original constructor:

JS Copy

```
function Point(x, y) { this.x = x; this.y = y; }
const BoundPoint = Point.bind(null, 10);
const p = new BoundPoint(20);
p.x; p.y;              // ⇒ 10 20  (10 baked in, 20 supplied)
p instanceof Point;    // ⇒ true
```

`call(null)` / `apply(null)` **don't always give** `null` `this`. In sloppy mode, passing `null` or `undefined` as `thisArg` substitutes the global object; only in strict mode does `this` actually stay `null` / `undefined`. That's why `Math.max.apply(null, nums)` is safe (it ignores `this`) but relying on `this === null` inside a sloppy function is a bug.

Currying and `fn.length`. Default parameters and rest params change `.length` (they're not counted), so a naive `curry` that keys off arity can stop collecting too early. If a function uses `(a, b = 1, ...rest)`, its `.length` is `1` — the curry helper will invoke after just one argument. Mention this limitation; it shows you understand *why* the arity trick works.

QUICK CHECK — no marks, just for you.

1. What does this log?

JAVASCRIPT Copy

```
const person = {
  fullName(city, country) {
    return `${this.first} ${this.last} from ${city}, ${country}`;
  },
};
```

- ◯ Ada Lovelace from undefined, undefined
- ◯ undefined undefined from Paris, France
- ◯ Ada Lovelace from Paris, France
- ◯ Ada Lovelace from ["Paris", "France"]

Check answer

2. What does this log?

JAVASCRIPT Copy

```javascript
function multiply(a, b) {
  return a * b;
}
const triple = multiply.bind(null, 3);
console.log(triple(5));
```

- ◯ 15
- ◯ 8
- ◯ [Function: bound multiply]
- ◯ NaN

Check answer

3. What does this log?

JAVASCRIPT Copy

```
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return (...next) => curried.apply(this, [...args, ...next]);
  };
}
function sum3(a, b, c) {
  return a + b + c;
}
const cs = curry(sum3);
console.log(cs(1)(2, 3));
```

- ◯ undefined
- ◯ 6
- ◯ [Function: curried]
- ◯ 3

Check answer

‹ Previous
The this Keyword

Next ›
Higher-Order Functions & Functional Patterns

LEARN MORE

ARTICLES & DOCS

Function.prototype.bind (MDN)
MDN

JavaScript Interview Kit

VIDEOS



bind, call & apply explained

YouTube



call, apply & bind step by step

YouTube



call vs apply vs bind (interview)

YouTube

Hand-picked references for this lesson.